Improving and Optimizing NeCT for NeCTv2

Master's Thesis

Department of Computer Science / Department of Physics
Norwegian University of Science and Technology (NTNU)

Kristian Gautefall Carlenius

Supervisor: *Ole Jakob Mengshoel & Dag Werner Breiby*

April 10, 2026

Abstract

[TODO: Writing the abstract last, summarise motivation, methods, key results, and conclusions aiming for 250 words.]

Acknowledgements

[TODO: Acknowledgements.]

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Understanding processes that unfold inside materials is one of the persistent challenges in both natural sciences and engineering. In geosciences, questions about how fluids move through rock pores, how minerals dissolve and precipitate under changing chemical conditions, and how rock deforms under mechanical stress all require direct observation of the three-dimensional internal structure as it changes over time. X-ray computed tomography has become the principal tool for this kind of observation, enabling non-destructive imaging of an internal structure at resolutions down to the micrometre scale. What remains difficult is time.

A conventional micro-CT scan of a rock sample takes on the order of one hour to acquire a single three-dimensional image. For a process such as brine displacing oil in a pore network, or a salt grain dissolving as a fluid front advances, the relevant events may unfold on timescales of seconds. The gap between what is physically happening and what the direct imaging can resolve in time is many orders of magnitude. Bridging this gap requires smarter acquisition schemes or reconstruction algorithms that extract more information from fewer measurements. Ideally some combination of both.

1.1 Background and Motivation

The standard approach to four-dimensional CT, denoted 4D-CT for the three spatial dimensions plus one dimension of time, is to acquire a sequence of independent full three-dimensional scans and reconstruct each independently. The temporal resolution of this approach is bounded below by the time required to complete one full 360ř rotation of the sample, which for laboratory instruments is governed by the photon flux of the X-ray source, the required signal-to-noise ratio per projection, and the number of angular positions needed for an artefact-free reconstruction. None of these constraints are easily relaxed without compromising spatial resolution or introducing reconstruction artefacts.

Reducing the number of projections per time step allows more time steps to be packed into a fixed total acquisition time, but with fewer projections the harder an accurate reconstruction becomes. Classical reconstruction algorithms such as the Feldkamp–Davis–Kress (FDK) algorithm and iterative methods with total variation regularisation handle sparse-view data poorly. FDK produces severe streak artefacts when the Shannon–Nyquist sampling criterion is not met, while total variation regularisation suppresses streaks at the cost of contrast reduction and loss of fine features. Neither algorithm uses information from other time steps to improve a given frame, so the temporal and spatial resolution trade-off is handled entirely within each independent reconstruction.

Machine learning approaches have been applied to sparse-view CT with considerable success on static images. Convolutional networks trained to map degraded FDK reconstructions to cleaner images can suppress artefacts effectively, but require large paired training datasets assembled in advance. More fundamentally, they have no internal model of the measurement physics, which means they are prone to generating plausible-looking but physically incorrect features when the test sample differs from the training distribution. For scientific imaging, where the correctness of individual pore-level features may carry direct physical consequences, this is a significant liability.

Implicit neural representations (INRs) offer a different approach. Rather than learning a mapping from degraded to clean images on a population of examples, an INR is fitted to the measurements of a single specific sample using the known physics of the CT measurement process as a differentiable forward model. The network receives no prior training data and is optimised from scratch for each scan, constrained to produce predictions that are consistent with the measured projections under Beer–Lambert's law. This instance-specific approach eliminates the hallucination problem and makes the reconstruction inherently physics-informed.

Neural Computed Tomography (NeCT), introduced by Friis et al. [2025], is the first algorithm to extend this INR-based approach to fully four-dimensional CT. Rather than reconstructing each time step independently, NeCT represents the entire dynamic experiment as a single continuous function $\mu(\mathbf{r}, t)$ using a hash-encoded neural network, trained jointly on all projections collected over the full duration of the scan. This holistic treatment allows the reconstruction to exploit angular information from all time steps simultaneously, enabling temporal resolutions approaching the time between individual projections with standard laboratory instruments.

The NeCT paper established the viability of the approach through experiments on both synthetic dynamic datasets and a real imbibition experiment on Bentheimer sandstone, demonstrating that individual pore-filling events and salt dissolution processes on timescales of 10 to 30 seconds could be directly observed with laboratory

equipment. However, the paper treated the encoder configuration as a fixed design choice and evaluated the model under a single maximally large configuration, measuring reconstruction quality as the primary outcome without considering computational cost. The configuration adopted requires GPU memory in excess of 60 GB, restricting training to the most expensive research-grade hardware available, and training times on the order of tens of hours further limit the practical applicability of the method.

This thesis builds directly on NeCT with the aim of reducing the computational cost and increasing functionality. It investigates three questions that the original paper left open: whether the encoder configuration can be reduced in size without a proportional loss of reconstruction quality, whether alternative encoder architectures offer a better trade-off between computational cost and accuracy, and whether the NeCT forward model can be extended to handle continuously acquired data in which the sample rotates without stopping between exposures.

1.2 Research Objectives

The work presented in this thesis is organised around three concrete research objectives, each corresponding to one experimental chapter.

The first objective is to characterise the relationship between encoder capacity and reconstruction quality in the NeCT QuadCubes architecture. The NeCT paper used the largest feasible configuration and trained for as long as possible, treating quality as the only optimisation target. This thesis instead asks whether there is a meaningful quality-versus-cost Pareto frontier, configurations that are substantially cheaper in memory and training time while achieving competitive reconstruction quality. Understanding this frontier is a prerequisite for using NeCT on hardware that is more widely accessible than an A100 or H100 GPU, and for workflows where reconstructions must be produced quickly enough to guide ongoing experiments.

The second objective is to evaluate alternative encoder architectures for the 4D hash encoding problem. The QuadCubes design uses four 3D multiresolution hash encoders covering all four possible 3-element subsets of the space-time coordinates. This is one point in a larger design space bounded by a single 4D encoder on one side, which suffers from a severe hash collision problem, and six 2D encoders covering all coordinate pairs on the other, which decouple the coordinates too strongly and require the MLP to perform a heavier integration task. This thesis evaluates several alternative architectures across this design space, including variations in encoder dimensionality, decoder architecture, and the use of dense encoders for the temporal dimensions.

The third objective is to extend the NeCT forward model to continuous rotation scanning, in which the detector shutter remains open throughout the acquisition and

each recorded projection integrates the sample over an angular interval rather than capturing it at a single angle. This mode of operation can increase the information collected per unit time relative to step-and-shoot scanning, at the cost of producing geometrically blurred projections. The thesis derives the modified Beer–Lambert forward model that accounts for this integration and evaluates whether it correctly compensates for the blurring and whether it enables the reconstruction of processes that occur within a single inter-projection interval.

1.3 Thesis Structure

Chapter 2 develops the theoretical background needed to understand the methods. It covers the fundamentals of neural networks and the multilayer perceptron, convolutional approaches to CT reconstruction and their limitations, and the theory of implicit neural representations including Neural Radiance Fields, positional encodings, and the multiresolution hash encoding. The second half of the background chapter covers X-ray CT from first principles, including Beer–Lambert's law, cone-beam geometry, classical reconstruction algorithms, and the challenges of 4D-CT and sparse-view acquisition. The chapter concludes with a detailed description of the NeCT algorithm itself, including both the static 3D model and the dynamic QuadCubes model, the training pipeline, and the hybrid golden-section sampling scheme.

Chapter 3 addresses the first research objective. It derives the memory cost of the baseline NeCT configuration, identifies the GPU hardware constraints that follow from it, and presents a systematic sweep of encoder size parameters including the hashmap capacity, number of levels, and features per level. Reconstruction quality and training time are measured jointly across all configurations, and the effect of allocating encoder capacity non-uniformly across the four encoders is also examined.

Chapter 4 addresses the second research objective. It explores the design space of encoder architectures by starting from the QuadCubes baseline and varying the number, dimensionality, and type of encoders. Alternative decoder architectures based on transformers and U-Nets are also evaluated, as is the use of dense (collision-free) encoders for the temporal dimensions.

Chapter 5 addresses the third research objective. It explains the physical difference between step-and-shoot and continuous rotation scanning, derives the K-step averaged forward model, and presents experimental results on both a static validation dataset and a dynamic dataset.

Chapter 6 summarises the contributions and findings of the thesis and identifies directions for future work.

Chapter 2

Background

2.1 Neural Networks

A neural network is a parameterised function whose internal parameters are adjusted iteratively until the function approximates a desired input-output relationship. The basic computational unit is the neuron, which computes a weighted sum of its inputs and passes the result through a nonlinear activation function. By stacking many neurons into layers and connecting layers sequentially, a network can represent increasingly complex relationships between its inputs and outputs.

The fundamental architecture used throughout this work is the *multilayer perceptron* (MLP), sometimes called a fully-connected or feedforward network. An MLP consists of an input layer, one or more hidden layers, and an output layer. Each layer ℓ applies a learned affine transformation followed by a nonlinear activation function:

$$\mathbf{h}^{(\ell)} = \sigma\Big(W^{(\ell)}\,\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}\Big), \tag{2.1}$$

where $W^{(\ell)}$ is the weight matrix, $\mathbf{b}^{(\ell)}$ is the bias vector of layer ℓ, and σ denotes the activation function applied element-wise. The input to the first layer is the network input $\mathbf{h}^{(0)} = \mathbf{x}$, and the output of the final layer L is the prediction $\hat{y} = \mathbf{h}^{(L)}$.

The composition of L such layers defines a function $f_\theta : \mathbb{R}^n \to \mathbb{R}^m$, where θ collects all learnable parameters $\{W^{(\ell)}, \mathbf{b}^{(\ell)}\}_{\ell=1}^{L}$. By the universal approximation theorem, an MLP with at least one hidden layer and a nonlinear activation can approximate any continuous function on a compact domain to arbitrary precision given sufficient capacity [Cybenko, 1989, Hornik et al., 1989].

Common activation functions include sigmoid $\sigma(z) = (1 + e^{-z})^{-1}$ and hyperbolic tangent $\sigma(z) = \tanh(z)$, both of which saturate for large inputs and can cause vanishing gradients in deep networks. The rectified linear unit (ReLU), $\sigma(z) = \max(0, z)$, largely solved this problem and became the default choice [Nair and Hinton, 2010]. A known failure mode of ReLU is that neurons whose pre-activation

is consistently negative receive no gradient and become permanently inactive. The Leaky ReLU addresses this by allowing a small gradient for negative inputs:

$$\sigma(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0, \end{cases} \tag{2.2}$$

where $\alpha \ll 1$ is a small constant. For applications requiring the network to represent signals with significant high-frequency content, periodic activations such as sinusoidal representation networks (SIREN, $\sigma(z) = \sin(z)$) have been proposed [Sitzmann et al., 2020].

Training a network means finding the parameter vector θ that minimises a scalar loss function $\mathcal{L}(\theta)$ measuring the discrepancy between predictions and observations. For regression tasks the two most common choices are the mean squared error (L2 loss) and the mean absolute error (L1 loss):

$$\mathcal{L}_{\text{L2}} = \frac{1}{N} \sum_{i=1}^{N} \left(\hat{y}_i - y_i\right)^2, \tag{2.3}$$

$$\mathcal{L}_{\text{L1}} = \frac{1}{N} \sum_{i=1}^{N} \left|\hat{y}_i - y_i\right|. \tag{2.4}$$

L1 loss is less sensitive to outliers than L2 and is often preferred when the measurements contain occasional corrupted samples. Parameters are updated by gradient descent, where at each step θ is adjusted in the direction of steepest descent of $\mathcal{L}$. The gradient $\nabla_\theta \mathcal{L}$ is computed by backpropagation, which applies the chain rule layer by layer from the output back to the input [Rumelhart et al., 1986]. In practice the gradient is estimated from a random mini-batch of B data points at each step rather than from the full dataset. The Adam optimiser [Kingma and Ba, 2015], which maintains per-parameter running estimates of the first and second gradient moments, is the standard choice for training coordinate networks.

2.2 Convolutional Neural Networks and Image-to-Image Architectures

While the MLP treats every input element independently, images have strong local structure. Nearby pixels tend to be similar, and the same local pattern such as an edge or a texture can appear anywhere in the image. Convolutional neural networks (CNNs) exploit this by replacing the dense weight matrices of an MLP with learned local filters that slide across the spatial extent of the input [LeCun et al., 1998].

A discrete 2D convolution of an input feature map $\mathbf{X} \in \mathbb{R}^{H \times W}$ with a filter

$\mathbf{K} \in \mathbb{R}^{k \times k}$ produces an output feature map $\mathbf{Y}$ whose entries are

$$Y_{i,j} = \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} K_{p,q}\, X_{i+p,\, j+q}. \tag{2.5}$$

The same kernel $\mathbf{K}$ is applied at every spatial position, so the number of learnable parameters depends only on the kernel size rather than the image dimensions. Each output unit depends on only a $k \times k$ neighbourhood of the input, called its receptive field. Stacking multiple convolutional layers increases the effective receptive field of deeper units, allowing the network to integrate information across progressively larger regions. Pooling layers downsample the feature maps, introducing robustness to small spatial shifts.

U-Net

The U-Net architecture [Ronneberger et al., 2015] is a widely used CNN for image-to-image tasks. It consists of a contracting encoder path that progressively halves spatial resolution while doubling the number of feature channels, and a symmetric expanding decoder path that restores the original resolution using transposed convolutions. Skip connections concatenate feature maps from the encoder to the corresponding decoder stage, allowing the network to recover fine spatial detail that would otherwise be lost during downsampling. In CT reconstruction, U-Nets are typically used to map a degraded FDK reconstruction to a cleaner image [Jin et al., 2017]. They have also been applied in the sinogram domain, where a network learns to predict missing projections from a sparse-view sinogram before passing the completed data to a conventional reconstruction algorithm [Hu et al., 2021].

Transformers

Transformers, originally developed for natural language processing, use a self-attention mechanism that allows every element in a sequence to attend to every other element. Applied to images, a vision transformer divides the input into fixed-size patches and treats them as a sequence of tokens. Self-attention captures long-range dependencies directly, unlike convolutions which build up spatial context gradually through stacking. This global receptive field makes transformers attractive for problems where features at distant spatial locations carry relevant information about each other. However, the quadratic memory cost of self-attention with respect to the number of tokens limits the resolution at which transformers can be applied to volumetric data.

Limitations for CT Reconstruction

Despite their strong empirical performance on static reconstruction tasks, CNN- and transformer-based approaches share a set of structural limitations. They must be trained on a dataset of paired examples before deployment, so assembling training data for a new experimental configuration is expensive. Once trained, the network is committed to that configuration and applying it to a new one requires retraining. These networks have no internal model of the measurement process and learn only a statistical mapping from the training distribution. When the test image differs from that distribution, as is common in novel scientific experiments, the network can produce physically incorrect features with high confidence [Patwari et al., 2023]. They also operate on discrete pixel or voxel grids, so the spatial resolution is fixed at training time. Representing a continuously evolving three-dimensional object at arbitrary spatial and temporal coordinates is not a natural fit for these frameworks.

2.3 Implicit Neural Representations

An implicit neural representation (INR), sometimes called a coordinate network or neural field, is a neural network trained to represent a signal as a continuous function of its coordinates. Formally, an INR is a parameterised function

$$f_\theta : \mathbb{R}^n \longrightarrow \mathbb{R}^m, \tag{2.6}$$

where θ denotes the network weights, the input is a coordinate vector such as a spatial position or a position-time pair, and the output is the signal value at that coordinate. The signal is implicit in the sense that it is never stored as a discrete grid of samples. Instead, the continuous function f_θ is the representation itself, and individual values are obtained by evaluating the network at any desired coordinate.

A 3D volume stored as a voxel grid of resolution N^3 requires memory proportional to N^3. An INR stores only its network weights, whose count is independent of the desired output resolution. The same network architecture can be queried at 128^3 or 1024^3 spatial positions without any change to its parameter count, making INRs inherently resolution-agnostic.

An INR is instance-specific, meaning it is trained from scratch for each individual signal, using only the measurements of that signal as supervision. No external datasets or prior training are needed. The optimisation proceeds by defining a differentiable forward model that maps the network's predicted field to the expected measurements, then minimising the discrepancy between those predictions and the actual observations. Because the physics of the measurement process is embedded directly in the optimisation loop, the reconstruction is constrained to be consistent

with the known forward model. This prevents the generation of features that are inconsistent with the measurements, which is a significant failure mode of trained CNN approaches [Patwari et al., 2023].

A naive INR, one in which an MLP receives raw coordinate inputs, struggles to represent signals with significant high-frequency content such as sharp material boundaries. This is a consequence of spectral bias, where gradient-based training causes MLPs to learn low-frequency components of a target function far faster than high-frequency ones [Rahaman et al., 2019]. Analysed through the neural tangent kernel (NTK), a standard coordinate-based MLP corresponds to a kernel whose eigenvalue spectrum decays rapidly with frequency, which prevents convergence to high-frequency components within a practical number of optimisation steps [Tancik et al., 2020].

The standard remedy is to map the raw input coordinates $\mathbf{x} \in \mathbb{R}^n$ through a positional encoding $\gamma : \mathbb{R}^n \to \mathbb{R}^d$ before passing them to the MLP, where $d \gg n$:

$$f_\theta(\mathbf{x}) = \mathrm{MLP}_\theta(\gamma(\mathbf{x}))\,. \tag{2.7}$$

The sinusoidal positional encoding popularised by NeRF [Mildenhall et al., 2021] maps each coordinate to a sequence of sines and cosines at exponentially increasing frequencies:

$$\gamma(\mathbf{x}) = \Big(\sin(2^0\pi\mathbf{x}),\ \cos(2^0\pi\mathbf{x}),\ \ldots,\ \sin(2^{L-1}\pi\mathbf{x}),\ \cos(2^{L-1}\pi\mathbf{x})\Big)\,, \tag{2.8}$$

where L is the number of frequency octaves. Tancik et al. [2020] showed using NTK theory that this encoding transforms the effective kernel into a stationary one with a tunable bandwidth, directly addressing spectral bias. An alternative is to use random Fourier features $\gamma(\mathbf{x}) = [\sin(\mathbf{Bx}), \cos(\mathbf{Bx})]$, where each row of $\mathbf{B}$ is sampled from a Gaussian $\mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$ with bandwidth σ chosen to match the signal's frequency content.

Both sinusoidal encodings and random Fourier features are fixed and not updated during training. A more capable alternative is to learn the encoding jointly with the MLP. The multiresolution hash encoding of Müller et al. [2022] pursues this by storing trainable feature vectors in a set of hash tables indexed by the input coordinates at multiple spatial resolutions. This shifts much of the representational work from the MLP to the encoding tables, allowing a smaller and faster MLP to achieve equal or better reconstruction quality. The hash encoding is described in detail in section 2.3.2.

2.3.1 Neural Radiance Fields (NeRF)

Neural Radiance Fields (NeRF), introduced by Mildenhall et al. [2021], demonstrated that a complex 3D scene could be fully represented by the weights of a small MLP trained on a set of 2D photographs taken from known camera positions, and that this representation was sufficient to synthesise photorealistic images from arbitrary new viewpoints. What made NeRF influential beyond its original application was the mathematical structure it established. It created a differentiable forward model connecting a continuous volumetric field to 2D observations via a ray integral, trained end-to-end by minimising the discrepancy between simulated and measured projections. This same structure, a continuous scalar field recovered through a physics-based differentiable renderer, is directly applicable to tomographic reconstruction.

NeRF represents a static scene as a continuous 5D function

$$F_\Theta : (\mathbf{x},\, \mathbf{d}) \longrightarrow (\mathbf{c},\, \sigma), \tag{2.9}$$

where $\mathbf{x} = (x, y, z) \in \mathbb{R}^3$ is a spatial location, $\mathbf{d} \in \mathbb{S}^2$ is a unit viewing direction, $\mathbf{c} = (r, g, b)$ is the emitted RGB colour, and $\sigma \geq 0$ is the volume density at $\mathbf{x}$. The density represents the differential probability that a ray is absorbed at $\mathbf{x}$ and is predicted from position only to enforce multi-view consistency. The colour depends on both position and direction, allowing NeRF to represent view-dependent effects such as specular reflections.

To render the colour at a pixel, a camera ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ is cast from the camera origin $\mathbf{o}$ into the scene. The expected colour $C(\mathbf{r})$ accumulated along the ray between near and far bounds t_n and t_f is given by the volume rendering integral [Kajiya and Von Herzen, 1984]:

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t)\, \sigma(\mathbf{r}(t))\ \mathbf{c}(\mathbf{r}(t), \mathbf{d})\ \mathrm{d}t, \tag{2.10}$$

where the accumulated transmittance $T(t) = \exp\Big(-\int_{t_n}^{t} \sigma(\mathbf{r}(s))\mathrm{d}s\Big)$ is the probability that the ray travels from t_n to t without being absorbed. The integral is approximated using stratified sampling, dividing the interval $[t_n, t_f]$ into N bins with one sample t_i drawn uniformly from each [Mildenhall et al., 2021]:

$$\hat{C}(\mathbf{r}) = \sum_{i=1}^{N} T_i \left(1 - e^{-\sigma_i \delta_i}\right) \mathbf{c}_i, \qquad T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right), \tag{2.11}$$

where $\delta_i = t_{i+1} - t_i$ is the distance between adjacent samples. This expression is fully differentiable with respect to the MLP parameters Θ.

NeRF is trained by minimising the squared photometric error between the

rendered colour and the observed pixel colour over all training rays, with no 3D supervision required. The structural parallel with CT reconstruction is close, as summarised in Table 2.1. Setting $\sigma(\mathbf{x}) \equiv \mu(\mathbf{x})$ and dropping the colour output reduces the NeRF forward model to Beer–Lambert's law. The CT reconstruction problem can therefore be addressed using the same INR machinery as NeRF, substituting the Beer–Lambert integral for the volume renderer and choosing an encoding suited to the properties of tomographic data.

Table 2.1: Structural analogy between NeRF and attenuation-contrast CT.

Concept	**NeRF**	**CT**
Recovered field	Volume density $\sigma(\mathbf{x})$	Attenuation coefficient $\mu(\mathbf{x})$
Measurements	Pixel colours $C(\mathbf{r})$	Detector intensities I_j
Forward model	Volume rendering integral	Beer–Lambert law
Ray sampling	Stratified random sampling	Equidistant ray marching
Loss function	L2 photometric error	L1 intensity error
Input encoding	Sinusoidal $\gamma(\mathbf{x})$	Multiresolution hash grid

2.3.2 Multiresolution Hash Encoding

The sinusoidal positional encoding resolves the spectral bias problem but places all of the representational work on the MLP. Every gradient step updates every weight in the network, and convergence is slow regardless of how well the encoding lifts the input coordinates. The multiresolution hash encoding of Müller et al. [2022] addresses this by distributing the work between the MLP and a set of small trainable lookup tables, allowing the MLP itself to be made much smaller. The result is a speedup of several orders of magnitude over sinusoidal-encoded NeRF at equal or better reconstruction quality [Müller et al., 2022].

The encoding maintains L independent levels, each corresponding to a different spatial resolution. At each level ℓ a grid of resolution N_ℓ is defined, where the resolutions grow as a geometric progression between a coarsest resolution $N_{\min}$ and a finest resolution $N_{\max}$:

$$N_\ell = \left\lfloor N_{\min} \cdot b^\ell \right\rfloor, \qquad b = \exp\left(\frac{\ln N_{\max} - \ln N_{\min}}{L-1}\right). \tag{2.12}$$

Each level stores up to T trainable feature vectors of dimension F, arranged in a flat array. At coarse levels, where the grid has fewer than T vertices, each vertex maps to a unique array entry (a 1:1 dense mapping). At finer levels, where the number of grid vertices exceeds T, the array is treated as a hash table and each vertex is mapped to an entry by a spatial hash function $h : \mathbb{Z}^d \to \mathbb{Z}_T$. The total number

of trainable encoding parameters is therefore bounded by $T \cdot L \cdot F$ but is typically somewhat smaller, because the coarsest levels store only as many entries as they have grid vertices rather than the full T. Typical values used in NeCT are $L = 23$ levels, $T = 2^{23}$ entries, $F = 4$ features per entry, and $N_{\min} = 16$ [Friis et al., 2025].

Given an input coordinate $\mathbf{x} \in \mathbb{R}^d$, the encoding proceeds independently at each level ℓ. The coordinate is scaled by that level's grid resolution to identify the enclosing voxel, and the 2^d integer corner vertices of that voxel are determined. Each corner is mapped to an array index via either the dense mapping at coarse levels or the hash function at fine levels, and the corresponding F-dimensional feature vectors are retrieved. These 2^d feature vectors are then linearly interpolated according to the fractional position of $\mathbf{x}$ within its enclosing voxel, using trilinear interpolation in 3D or the 4D analogue in space-time. The interpolated F-dimensional vector is the output of level ℓ. The outputs from all L levels are concatenated to form the full encoded representation $\mathbf{y} \in \mathbb{R}^{L \cdot F}$, which is then passed to the MLP:

$$\hat{\mu}(\mathbf{x}) = \mathrm{MLP}_\Phi(\mathbf{y}(\mathbf{x};\, \theta))\,, \qquad \mathbf{y} = \Big[\,\mathrm{interp}_0(\mathbf{x};\, \theta_0),\ \mathrm{interp}_1(\mathbf{x};\, \theta_1),\ \ldots,\ \mathrm{interp}_{L-1}(\mathbf{x};\, \theta_{L-1})\,\Big]\,. \tag{2.13}$$

Crucially, linear interpolation is differentiable, so gradients flow back through the concatenation and interpolation to the feature vectors θ_ℓ at each level, updating only the 2^d entries involved in each forward pass. For a 3D grid this is 8 entries per level per sample rather than every weight in the MLP. This sparse gradient update is the primary reason the encoding converges so much faster than a purely MLP-based representation [Müller et al., 2022].

At fine resolution levels, multiple grid vertices inevitably map to the same hash table entry. Unlike classical hash tables which resolve such collisions through chaining or probing, the multiresolution hash encoding handles them implicitly. When two vertices collide, their gradients accumulate in the same feature vector during training. The gradients from the region with the highest loss dominate the update, so the shared vector is pulled toward representing the most important detail. The MLP then uses context from the other $L - 1$ levels, which encode the same location at different resolutions with different hash functions, to disambiguate colliding entries at query time. No explicit data-structure updates such as pruning or splitting are needed at any point during training [Müller et al., 2022], which maps well to GPU parallelism.

Because hash table lookups are $\mathcal{O}(1)$ with no control-flow branching, all L levels can be evaluated in parallel on a GPU. The small MLP that follows requires far fewer floating-point operations than the large MLP needed by sinusoidal encodings. Together these properties enable training to convergence in seconds rather than hours on a single GPU, a speedup of roughly two to three orders of magnitude over the

original NeRF implementation [Müller et al., 2022].

The first application of multiresolution hash encoding to CT reconstruction was Neural Attenuation Fields (NAF) by Zha et al. [2022]. NAF parameterises the 3D attenuation coefficient field $\mu(\mathbf{x})$ as a hash-encoded MLP and trains the network by minimising the discrepancy between simulated and measured cone-beam projections using Beer–Lambert's law as the forward model, with no external training data required. NAF showed that the hash encoding's advantages transfer directly from NeRF to CT and outperform both sinusoidal-encoded INRs and iterative algorithms at a fraction of the computational cost [Zha et al., 2022].

2.4 X-ray Computed Tomography

2.4.1 Physical Principles

X-ray computed tomography reconstructs the three-dimensional internal structure of an object from a set of its two-dimensional projections. The physical quantity being reconstructed is the linear attenuation coefficient $\mu(\mathbf{r})$, a material-dependent scalar field defined at every point $\mathbf{r} = (x, y, z)$ in the object that describes how strongly the local material absorbs or scatters X-ray photons. Its SI unit is m^{-1} and it depends on both the material density and the photon energy. In attenuation-contrast CT the contrast between different materials arises entirely from differences in μ.

When a monochromatic X-ray beam of initial intensity I_0 travels through a heterogeneous material along a straight ray path, the transmitted intensity I is governed by Beer–Lambert's law:

$$I = I_0 \exp\Big(- \int_{\text{ray}} \mu(\mathbf{r}(t))\, \mathrm{d}t \Big), \tag{2.14}$$

where the integral is taken along the parameterised ray path $\mathbf{r}(t) = \mathbf{o} + t\,\mathbf{d}$, with $\mathbf{o}$ the source position and $\mathbf{d}$ a unit direction vector. Taking the negative logarithm of the transmitted-to-incident intensity ratio gives the line integral of μ along the ray, also called the projection value p:

$$p = -\ln\Big(\frac{I}{I_0}\Big) = \int_{\text{ray}} \mu(\mathbf{r}(t))\, \mathrm{d}t. \tag{2.15}$$

CT reconstruction is the inverse problem of recovering $\mu(\mathbf{r})$ from a finite set of such projection values. Beer–Lambert's law assumes the X-ray beam is effectively monochromatic and that scattering is negligible, both of which are standard assumptions in laboratory micro-CT analysis [Kak and Slaney, 2001].

In cone-beam CT (CBCT), a point X-ray source illuminates the entire sample

simultaneously and the transmitted intensities are recorded on a flat 2D area detector. This is the geometry of standard laboratory micro-CT instruments. A single projection, also called a radiograph, is the 2D image $\{I_j\}$ recorded by the detector at one angular position of the rotating sample, where j indexes the detector pixels. As the sample rotates, successive projections sweep a full 360ř arc, and the complete set of projections forms the sinogram.

In practice the continuous Beer–Lambert integral is approximated by a discrete sum over K sample points along the ray:

$$p_j \approx \sum_{k=1}^{K} \mu(\mathbf{r}_k)\, \Delta t, \tag{2.16}$$

where Δt is the sampling step size. CT reconstruction then amounts to inverting the system of equations $\{p_j = \mathcal{A}\mu\}$, where $\mathcal{A}$ is the forward projection operator. The system is highly underdetermined when the number of projections is small relative to the number of voxels, making the inverse problem ill-posed and requiring regularisation or prior information for a stable solution.

2.4.2 Classical Reconstruction Algorithms

Reconstruction algorithms fall into two broad families. Analytical methods derive a closed-form inversion formula from the Fourier slice theorem, while iterative methods formulate reconstruction as an optimisation problem.

The Feldkamp–Davis–Kress (FDK) algorithm [Feldkamp et al., 1984] is the standard workhorse for cone-beam CT. It approximates the 3D cone-beam geometry by treating each detector row as an independent tilted fan-beam slice, applies a cosine-weighted ramp filter, and back-projects the filtered data into a 3D reconstruction volume. FDK is non-iterative and fast, but requires the number of projections N_ϕ to satisfy the Shannon–Nyquist criterion, approximately $N_\phi \geq \pi N/2$ for N pixels across the field of view [Kak and Slaney, 2001]. When this condition is violated, as it inevitably is in sparse-view CT, reconstructions suffer from characteristic streak artefacts.

Iterative methods formulate reconstruction as minimisation of a data fidelity term. The simultaneous algebraic reconstruction technique (SART) [Andersen and Kak, 1984] updates all voxels simultaneously using rays from one projection angle at each step. Ordered subsets SART (OS-SART) further accelerates convergence by processing projections in angularly diverse subsets [Du et al., 2017]. Adding total variation (TV) regularisation yields OS-SART-TV, which penalises the image gradient magnitude:

$$\mathcal{R}_{\mathrm{TV}}(\mu) = \sum_{\mathbf{r}} \left\| \nabla \mu(\mathbf{r}) \right\|_2 , \tag{2.17}$$

promoting piecewise-constant solutions. TV regularisation can suppress streak artefacts in sparse-view conditions, but introduces contrast reduction as the penalty discourages gradients everywhere, smoothing out fine features [Kak and Slaney, 2001].

Both FDK and OS-SART-TV were designed for static 3D CT. Neither algorithm can use information from other time steps to improve a given frame, so each reconstruction is treated independently.

2.4.3 4D-CT and Dynamic Imaging

4D-CT denotes acquisition and reconstruction of CT data from a sample that evolves over time, producing a time-resolved sequence of 3D images. In geosciences, 4D-CT is applied to study mechanical deformation of rocks, multiphase fluid flow, CO_2 mineralisation, and reactive dissolution [Withers et al., 2021]. The central challenge is achieving sufficient temporal resolution to capture the process of interest while maintaining the spatial resolution needed to observe pore-scale phenomena.

The temporal resolution of a conventional 4D-CT scan is limited by the time required to collect enough angular projections for a 3D reconstruction. In sequential scanning a full 360ř rotation is completed before moving to the next time step, giving a temporal resolution on the order of one hour for laboratory micro-CT. Reducing this requires fewer projections per time step, which degrades reconstruction quality. Several mitigation strategies exist, including incorporating prior information from a high-quality static scan [Chen et al., 2008, Myers et al., 2011], penalising the temporal derivative of μ to enforce smooth evolution [Goethals et al., 2022], and representing the object as a continuous function of space and time fitted to all projections simultaneously.

The photon flux of the X-ray source is a fundamental constraint. Synchrotron sources provide a beam brilliance 10–14 orders of magnitude higher than laboratory tubes [Withers et al., 2021], enabling sub-second exposures. Standard laboratory instruments are far more accessible but require longer exposures, limiting acquisition speed.

The choice of angular sampling scheme affects how much complementary information is gathered per projection. In equiangular scanning, projections are collected at equally spaced angles, which is well suited to static reconstruction but provides redundant angular coverage in dynamic experiments where consecutive projections are taken at similar angles. The golden-angle sampling scheme sets the angular increment to $\Delta\phi \approx 137.51$ř, distributing projections nearly optimally on a circle as more are accumulated [Kohler, 2004]. At any point during acquisition, the projections collected so far are as uniformly spread in angle as possible, which is useful

for time-resolved imaging where the effective number of projections per time step is small.

2.5 NeCT: Neural Computed Tomography

NeCT (Neural Computed Tomography) [Friis et al., 2025] is, to the best of the authors' knowledge, the first fully INR-based algorithm for holistic time-resolved 4D-CT reconstruction. It unifies the physical forward model of cone-beam CT with the representational power of hash-encoded implicit neural representations, treating the entire dynamic experiment, all projections from all time steps, as a single joint optimisation problem. The output is a continuous function $\hat{\mu}(\mathbf{r}, t)$, the estimated attenuation coefficient at any spatial coordinate $\mathbf{r}$ and any time t within the duration of the scan, from which volumetric images at arbitrary time points can be decoded by a forward pass through the network.

NeCT consists of two architecturally related but distinct models. A static 3D model for sparse-view single-frame CT reconstruction serves as the spatial backbone, and a dynamic 4D model called *NeCT QuadCubes* extends this to time-resolved reconstruction. Both share the same training loop and loss function, differing in the structure of the hash encoding and whether the time coordinate t is included as an input.

2.5.1 NeCT Pipeline Overview

The NeCT pipeline operates as a differentiable physics simulation. It takes the measured detector images as supervision and adjusts the network weights until the network's predictions are consistent with every recorded projection. The same four-step loop repeats for every training batch.

At the start of each step, a batch of B rays is sampled from the full set of recorded projections. Each ray $\mathbf{r}_j(t) = \mathbf{o}_j + t\,\mathbf{d}_j$ is defined by its source position $\mathbf{o}_j$ and unit direction $\mathbf{d}_j$, determined by the cone-beam geometry of the instrument and the angular position of the projection from which the ray originates. In the dynamic model, each ray also carries a timestamp t_j corresponding to the acquisition time of its projection. The measured intensity I_j at the corresponding detector pixel is retrieved as the supervision target.

Each ray is then marched through the object volume by sampling K equidistant points $\{\mathbf{r}_k\}_{k=1}^{K}$ along the ray within the sample bounding volume. NeCT uses an adaptive scheme in which K increases linearly from $K_{\text{init}} = 100$ to $K_{\max} = 1.5 \times \max(N_{\text{detector}})$ over the first 30% of training, where $\max(N_{\text{detector}})$ is the number of pixels along the longest detector axis [Friis et al., 2025]. Starting with fewer

points reduces computational cost in the early stages when the network is still poorly initialised, and progressively increasing the count ensures the final reconstruction is sampled finely enough to resolve sub-voxel features.

Each sampled point $(\mathbf{r}_k, t_j)$ is passed through the neural network to obtain the predicted attenuation coefficient $\hat{\mu}(\mathbf{r}_k, t_j)$. In the static model t_j is omitted. The predicted detector intensity for ray j is then computed by discretising Beer–Lambert's law:

$$\hat{I}_j = I_0 \exp\left(- \sum_{k=1}^{K} \hat{\mu}(\mathbf{r}_k,\, t_j)\, \Delta s \right), \tag{2.18}$$

where Δs is the step size between adjacent sample points. The predicted intensities are compared to the measured intensities using the L1 loss over the batch:

$$\mathcal{L} = \frac{1}{B} \sum_{j=1}^{B} \left| \hat{I}_j - I_j \right|. \tag{2.19}$$

Gradients of $\mathcal{L}$ with respect to all network parameters are computed by automatic differentiation and used to update the parameters via the Adam optimiser [Kingma and Ba, 2015]. The learning rate follows a schedule with a linear warm-up over the first 5 000 batches, after which it decays according to a cosine schedule to 1% of its peak value [Friis et al., 2025]. Base learning rates are 1×10^{-3} for static reconstructions and 2×10^{-4} for dynamic reconstructions. The batch size is 5×10^6 ray sample points per step.

Because the hash table entries are updated only for the small subset of grid vertices intersected by each batch of rays (just $2^3 = 8$ entries per level per ray for a 3D encoder), the effective gradient update is extremely sparse, which gives the hash encoding its computational efficiency.

2.5.2 Static 3D Reconstruction

The static NeCT model represents the attenuation coefficient as a continuous function of spatial coordinates only, $\hat{\mu} : \mathbb{R}^3 \to \mathbb{R}$, implemented as a hash-encoded MLP. It is architecturally derived from NAF [Zha et al., 2022] with a refined encoder configuration and training protocol.

The input spatial coordinate $\mathbf{r} = (x, y, z)$ is passed through a single 3D multiresolution hash encoder. The encoder uses a hashmap with $T = 2^{23}$ entries, $L = 23$ resolution levels, a base resolution of $N_{\min} = 16$, a growth factor of $b = 2$, and $F = 4$ features per entry [Friis et al., 2025]. The MLP has 4 hidden layers each with 128 neurons and Leaky ReLU activations, with a single linear output neuron producing the scalar $\hat{\mu}(\mathbf{r})$. The finest grid resolution $N_{\max}$ is calibrated to the detector resolution of the specific CT scan so that the finest hash grid voxels correspond approximately

to the physical voxel size.

NeCT was benchmarked using peak signal-to-noise ratio (PSNR) and the structural similarity index measure (SSIM) [Wang et al., 2004] on eleven standard 3D CT datasets from the Open SciVis Datasets project [Klacansky, 2017]. PSNR is defined as:

$$\mathrm{PSNR} = 10 \log_{10}\left(\frac{R^2}{\mathrm{MSE}(\hat{\mu},\, \mu^*)}\right), \tag{2.20}$$

measured in decibels, where R is the dynamic range of the attenuation values. SSIM captures perceptual similarity by comparing local statistics of luminance, contrast, and structure, ranging from 0 to 1 [Wang et al., 2004].

NeCT outperformed FDK, OS-SART-TV, and NAF on both metrics across all eleven datasets [Friis et al., 2025]. On the Stagbeetle dataset, NeCT achieved a PSNR of 45.6 dB and SSIM of 0.996, compared to 44.3 dB / 0.994 for NAF, 41.9 dB / 0.990 for OS-SART-TV, and 28.6 dB / 0.577 for FDK. Visually, NeCT reconstructions are free of streak artefacts and show sharp material boundaries, attributed in part to the spectral regularisation effect of the hash-encoded INR [Friis et al., 2025].

2.5.3 Dynamic 4D Reconstruction: NeCT QuadCubes

The dynamic NeCT model, named QuadCubes, extends the 3D INR to four dimensions by representing the attenuation field as a continuous function of both space and time, $\hat{\mu} : \mathbb{R}^3 \times \mathbb{R} \to \mathbb{R}$. The central design challenge is how to structure the hash encoding for a 4D input space. QuadCubes uses four independent 3D multiresolution hash encoders, each covering a different triplet of the four space-time dimensions:

$$\text{Encoder 1: } (x, y, z), \quad \text{Encoder 2: } (x, y, t), \quad \text{Encoder 3: } (x, z, t), \quad \text{Encoder 4: } (y, z, t). \tag{2.21}$$

This set covers all four possible 3-element subsets of $\{x, y, z, t\}$, so every pair of coordinates appears together in at least two encoders. Each encoder independently produces a feature vector of dimension $L \cdot F$, and the four vectors are concatenated to form the full MLP input:

$$\hat{\mu}(\mathbf{r}, t) = \mathrm{MLP}_\Phi\Big(\big[\gamma_{xyz}(\mathbf{r};\, \theta_1),\ \gamma_{xyt}(x, y, t;\, \theta_2),\ \gamma_{xzt}(x, z, t;\, \theta_3),\ \gamma_{yzt}(y, z, t;\, \theta_4)\big]\Big), \tag{2.22}$$

where $\gamma_{(\cdot)}$ denotes the trilinear-interpolated hash encoding in the indicated coordinate triplet and θ_i are the trainable feature vectors of encoder i. The MLP has the same architecture as the static model, with 4 hidden layers of 128 neurons and Leaky ReLU activations.

Each encoder operates in a well-conditioned 3D hash space where the collision rate remains manageable at micro-CT resolutions. The overlap between encoders,

where the spatial encoder `xyz` shares each of its three dimensions with one of the temporal encoders, provides redundant representations of spatial structure that assist collision disambiguation and give the MLP multiple views of the same location at different times. The rationale behind this particular factorisation, and how it compares to alternative designs, is the subject of Chapter 4.

A defining property of the QuadCubes model is that it is trained on all projections from the full dynamic experiment simultaneously. The network learns to distinguish between time-invariant structural features, which are supported by projections from all angles and all times, and dynamic features such as advancing fluid fronts, which are supported only by projections in their temporal vicinity. Static structures therefore converge to a sharp representation, while dynamic features converge with precision proportional to the rate at which they evolve relative to the projection acquisition rate. The temporal resolution is consequently scene-dependent, and for large, rapidly changing features a temporal resolution approaching the time between two successive projections is achievable [Friis et al., 2025].

Simulations using synthetic 4D datasets generated with PoreSpy [Gostick et al., 2019] showed that QuadCubes can resolve dynamics lasting as few as two projection intervals for large features, and approaches full resolution for events spanning ten or more projections. In the experimental Bentheimer sandstone dataset, individual pore-filling events with a characteristic time of approximately 10 s and salt dissolution events lasting approximately 30 s were directly observed [Friis et al., 2025].

2.5.4 Hybrid Golden-Section Sampling

A practical limitation of the pure golden-angle scheme described in section 2.4.3 is that consecutive projections are separated by 137.51ř, requiring the sample stage to travel through a large angle between exposures. In laboratory instruments this large step is time-consuming and can cause vibration. The hybrid golden-section scheme, introduced by Friis et al. [2025], addresses this. Within each full 360ř revolution a fixed number of equidistant projections are collected, but the starting angle of each revolution is offset from the previous one by the golden angle. This preserves rapid continuous rotation within each revolution while ensuring that successive revolutions are angularly offset, so that projections from different time points together cover the angular space as uniformly as possible.

This scheme provides progressive angular completeness, meaning that at any point in the experiment the projections collected so far are nearly optimally distributed in angle. For the spatial encoder `xyz`, this ensures gradient updates arrive from many different projection directions simultaneously rather than from a narrow angular range. For the temporal encoders, the golden offset means each new revolution

contributes projections at angles that are maximally distinct from those of all previous revolutions, allowing the model to distinguish the attenuation field at one time from the field at neighbouring times.

In the Bentheimer sandstone experiment, 1 400 projections were distributed across 14 revolutions of 100 projections each, with the golden offset applied between revolutions and alternating rotation direction to avoid tangling the inlet supply lines [Friis et al., 2025]. The entire 40-minute imbibition experiment was represented by a single QuadCubes model, decoded at arbitrary times to produce a continuous 3D movie of the brine invasion process.

Chapter 3

Encoder Size and Performance

The original NeCT paper [Friis et al., 2025] treated the encoder configuration as a fixed design choice and evaluated the model by training it to convergence with the largest possible configuration, measuring reconstruction quality as the sole outcome. This approach is appropriate when establishing that the method works at all, but it leaves open the question of whether the chosen configuration is efficient. In practice, the memory footprint of the QuadCubes model with the NeCT configuration is large enough to exclude all but the most expensive research-grade GPUs, and the training time for a single dynamic reconstruction is on the order of tens of hours. Both of these properties are significant barriers to practical use. This chapter investigates whether smaller encoder configurations can achieve competitive reconstruction quality in substantially less time and with less memory, and whether the relationship between encoder capacity and quality is smooth enough to permit principled trade-offs.

3.1 Experimental Setup

The Baseline Configuration and Its Cost

The encoder configuration used in the NeCT experiments is defined by the following hyperparameters, expressed in the notation of the `tiny-cuda-nn` hash grid implementation:

The $\log_2$ hashmap size of 23 sets the hash table capacity per level to $T = 2^{23} = 8\,388\,608$ entries. However, not every level requires the full T entries. With $b = 2$ and $N_{\min} = 16$, the resolution at level ℓ is $N_\ell = 16 \times 2^\ell$, giving N_ℓ^3 grid vertices. The coarsest levels, where $N_\ell^3 \leq T$, use a dense 1:1 mapping and store only N_ℓ^3 entries. This transition occurs at level 4, where $256^3 \approx 16.8 \times 10^6 > T$. Levels 0 through 3 are therefore dense and store a combined total of approximately 2.4×10^6 entries, while levels 4 through 22 (19 levels) each use the full T entries.

Table 3.1: Baseline QuadCubes hash encoder configuration from Friis et al. [2025].

Parameter	Value
Encoding type	`HashGrid`
Number of levels (L)	23
Features per level (F)	4
Log_2 hashmap size	23
Base resolution (N_{min})	16
Maximum resolution factor (b)	2

The total number of trainable encoding parameters per encoder is therefore:

$$\left(\sum_{\ell=0}^{3} N_\ell^3 + 19 \times T\right) \times F \approx (2.4 \times 10^6 + 159.4 \times 10^6) \times 4 \approx 647 \text{ million.} \tag{3.1}$$

The QuadCubes architecture uses four such encoders, giving a total encoding parameter count of approximately 2.6 billion. Stored at half precision (float16, 2 bytes per parameter), the hash tables alone occupy roughly 5.2 GB of GPU memory. Including the Adam optimiser, which stores a first-moment and a second-moment estimate for every parameter, the optimiser state for the hash tables adds another 10.4 GB. Together with the MLP weights, their optimiser states, intermediate activations, and the projection data loaded on device, the total GPU memory requirement for a QuadCubes training run with the baseline configuration **[TODO: insert measured peak VRAM figure]** exceeds the capacity of consumer and mid-range professional GPUs. In practice this restricts training to high-memory server GPUs such as the Nvidia A100 (80 GB HBM2e), H100 (80 GB HBM3), and H200 (141 GB HBM3e).

Training time for a full dynamic reconstruction with the baseline configuration on a single A100 GPU is 38 hours for the Bentheimer dataset.

Dataset

All experiments in this chapter use the Bentheimer dataset from the original NeCT paper. Using a single fixed dataset allows all configurations to be compared under identical conditions. **[TODO: Should use more than one dataset for comparison]**

Metrics

Reconstruction quality is evaluated using PSNR and SSIM as defined in section 2.5.2. In addition, mean absolute error (MAE) is reported:

$$\mathrm{MAE} = \frac{1}{|\Omega|} \sum_{\mathbf{r} \in \Omega} |\hat{\mu}(\mathbf{r}) - \mu^*(\mathbf{r})| \,, \tag{3.2}$$

where Ω is the set of voxels in the reconstruction volume and μ^* denotes the ground truth attenuation field. MAE is reported alongside PSNR and SSIM because it is in the same physical units as the attenuation coefficient and is therefore directly interpretable in terms of material contrast. All three metrics are computed on six 2D slices at three points in time, the first, last and middle timesteps of the scan.

Computational cost is characterised by two quantities: peak GPU memory consumption, measured as the maximum memory allocated during training, and total training time from initialisation. **[TODO: Might be better to measure convergence time rather than an arbitrary endpoint, but requires a definition of convergence such as "validation loss has not decreased by more than X% over Y epochs".]** All experiments are run separately, each using one A100 with 80 GB VRAM.

Varied Parameters

The encoder size is controlled by three hyperparameters that dominate the parameter count: the $\log_2$ hashmap size (equivalently, $\log_2 T$), the number of levels L, and the number of features per level F. The MLP width and depth are held fixed at 128 neurons and 4 hidden layers throughout this chapter, since the hash table dominates the parameter count by several orders of magnitude and changes to the MLP have negligible impact on memory and training time relative to changes in the encoder.

3.2 Effect of Total Parameter Count

Varying the Hashmap Size

The $\log_2$ hashmap size is the single parameter with the largest effect on both memory and parameter count, since the total number of encoding parameters scales linearly with $T = 2^{\log_2 T}$. Values of 17, 18, 19, 20, 21, 22, 23 were evaluated, with L and F held at their baseline values of 23 and 4.

Figure **??** shows PSNR and SSIM as a function of total parameter count for this sweep. **[TODO: Insert figure: x-axis = log-scale parameter count or log2_hashmap_size, y-axis = PSNR / SSIM, one curve each.]**

[TODO: Describe the trend observed in the figure without drawing conclusions.]

Varying the Number of Levels

The number of levels L affects both the total parameter count and the range of spatial frequencies that the encoder can represent simultaneously. With $b = 2$ and $N_{\min} = 16$, increasing L extends the encoder's coverage to finer spatial scales, but also adds $T \times F$ new parameters per additional level. Values of 17, 18, 19, 20, 21, 22, 23 were evaluated, with the hashmap size and features per level held at their baseline values.

Figure **??** shows reconstruction quality as a function of L. **[TODO: Insert figure and describe observed trend.]**

Varying Features Per Level

The number of features per level F controls the dimensionality of each interpolated feature vector and therefore the amount of information the encoder can store per spatial location at each resolution. Values of 2, 4 were evaluated. Changing F also changes the input dimension of the MLP, since the MLP receives a concatenated vector of dimension $4 \times L \times F$ for QuadCubes.

Figure **??** shows reconstruction quality as a function of F. **[TODO: Insert figure and describe observed trend.]**

Joint Parameter Sweep

To understand whether total parameter count is a reliable predictor of reconstruction quality regardless of which hyperparameter it comes from, all three sweeps above are plotted together against total encoding parameter count in Figure **??**. **[TODO: Insert joint figure and describe.]**

3.3 Effect on Reconstruction Time

The relationship between encoder size and reconstruction quality is only one dimension of the trade-off. A configuration with slightly lower PSNR but substantially faster training may be preferable for many applications, particularly when the bottleneck is GPU availability or when iterative experimental workflows require multiple reconstructions. Here we examine the joint relationship between training time and quality, asking which configurations lie on the Pareto front of quality versus time.

Figure **??** shows PSNR and SSIM as a function of training time for all configurations evaluated in section 3.2, with each point representing one configuration at convergence. **[TODO: Insert figure. The plot should resemble a quality-vs-time scatter, ideally with the Pareto front highlighted.]**

[TODO: Describe what the figure shows.]

Figure **??** shows the same configurations plotted against peak GPU memory consumption. **[TODO: Insert figure. Mark a horizontal line at the memory limits of common GPUs, e.g. 24 GB for RTX 3090/4090, 40 GB for A100 40GB, 80 GB for A100/H100 80GB.]**

[TODO: Describe which configurations fall within the memory budget of consumer or mid-range professional GPUs and whether those configurations achieve competitive reconstruction quality.]

Training Dynamics

Beyond the final converged quality, the rate at which a configuration converges matters for practical use. Figure **??** shows PSNR as a function of training time for a representative selection of configurations. **[TODO: Insert figure: multiple curves on a shared time axis.]**

[TODO: Describe the observed convergence behaviour.]

3.4 Precision and Accuracy Metrics

PSNR, SSIM, and MAE measure different aspects of reconstruction fidelity and do not always agree in their ranking of configurations. PSNR and MSE are sensitive to large absolute errors, which in a CT reconstruction typically correspond to errors at high-contrast boundaries. SSIM reflects the perceptual sharpness of material boundaries. MAE is less influenced by rare large errors and more reflective of the average accuracy across the volume.

Figure **??** shows PSNR, SSIM, and MAE for all configurations, sorted by total parameter count. **[TODO: Insert figure.]**

[TODO: Describe whether the three metrics agree in their ranking of configurations.]

For the specific application of multiphase flow imaging in porous media, the most physically meaningful metric depends on the intended downstream use. If the reconstruction will be segmented to extract pore-level geometry, SSIM and boundary sharpness are most relevant. If the goal is quantitative phase saturation estimation, MAE in absolute attenuation units is more directly informative. **[TODO: Add any domain-specific discussion once results are available.]**

3.5 Non-Uniform Encoder Sizing

All configurations considered so far assign the same hashmap size, number of levels, and features per level to all four encoders in QuadCubes. This uniform allocation may not be optimal. The spatial encoder `xyz` is responsible for representing the three-dimensional structure of the rock matrix, which is static throughout the experiment and benefits from many observations at many angles. The three temporal encoders `xyt`, `xzt`, and `yzt` are jointly responsible for representing the temporal evolution of the attenuation field, and they receive gradient updates only from projections at the corresponding time point. One might therefore expect that the spatial encoder benefits from higher capacity than each individual temporal encoder.

This section investigates non-uniform configurations in which the spatial encoder `xyz` is assigned a different capacity from the three temporal encoders. The temporal encoders are kept equal to each other throughout, so the configuration space is two-dimensional. Total parameter count is used as a budget, and configurations are evaluated at matched or near-matched totals to isolate the effect of the allocation.

Spatial-Heavy Allocation

In spatial-heavy configurations, a larger fraction of the total parameter budget is allocated to the `xyz` encoder, with the temporal encoders correspondingly reduced. Table 3.2 lists the specific configurations evaluated. **[TODO: Fill in the table.]**

Table 3.2: Non-uniform encoder configurations evaluated in section 3.5. All configurations use $L =$ **[TODO: value]** levels and $F =$ **[TODO: value]** features per level unless otherwise noted.

Configuration	`xyz` $\log_2 T$	Temporal $\log_2 T$	Total params	Notes
Uniform (baseline)	23	23	**[TODO:]**	Equal allocation
Spatial-heavy A	**[TODO:]**	**[TODO:]**	**[TODO:]**	
Spatial-heavy B	**[TODO:]**	**[TODO:]**	**[TODO:]**	
Temporal-heavy A	**[TODO:]**	**[TODO:]**	**[TODO:]**	
Temporal-heavy B	**[TODO:]**	**[TODO:]**	**[TODO:]**	

Figure **??** shows PSNR and SSIM for all non-uniform configurations compared to the uniform baseline at matched total parameter count. **[TODO: Insert figure.]**

[TODO: Describe the observed effect of allocation on reconstruction quality.]

Interpretation

[TODO: Describe any qualitative patterns in where reconstruction errors occur for spatial-heavy versus temporal-heavy configurations.]

Chapter 4

Encoder Architecture

Chapter 3 treated the number and dimensionality of encoders as fixed and varied only their capacity. This chapter takes the opposite approach, holding the total capacity per encoder approximately constant and varying the structural question of how many encoders to use, across which coordinate dimensions, and what type of decoder processes their output. The central tension in the design of the hash encoding for a 4D problem is between hash collision rate and dimensional coupling. A single high-dimensional encoder captures all interactions between coordinates directly but suffers from a severe collision problem at any practical memory budget. Many low-dimensional encoders reduce collisions but capture only low-order dependencies between coordinates, placing greater demands on the decoder to integrate the independent streams of information.

4.1 Collision Rate and the Design Space

The collision-free hashmap size for a hash encoder of input dimension d can be estimated by requiring the hash table capacity $T = 2^{\log_2 T}$ to be at least as large as the number of grid vertices at the finest resolution level. For a finest resolution $N_{\max}$ in each coordinate direction, the number of vertices is $N_{\max}^d$. Setting $T \geq N_{\max}^d$ gives the minimum collision-free $\log_2$ hashmap size as $d \cdot \log_2 N_{\max}$.

For a representative finest resolution of $N_{\max} = 2^{10} = 1024$ voxels per side, this gives:

$$\begin{aligned} d = 2: &\quad \log_2 T_{\text{free}} = 2 \times 10 = 20, \quad T_{\text{free}} \approx 10^6, \\ d = 3: &\quad \log_2 T_{\text{free}} = 3 \times 10 = 30, \quad T_{\text{free}} \approx 10^9, \\ d = 4: &\quad \log_2 T_{\text{free}} = 4 \times 10 = 40, \quad T_{\text{free}} \approx 10^{12}. \end{aligned}$$

At float16 precision with $L = 16$ levels and $F = 4$ features per level, the memory required for a single collision-free encoder grows from approximately 128 MB for

$d = 2$, to roughly 64 GB for $d = 3$, and to approximately 64 TB for $d = 4$. The $d = 4$ case is entirely infeasible with any current hardware, and even the $d = 3$ collision-free encoder would consume more memory than is available on any single GPU. In practice, all 3D and 4D configurations operate in the collision regime and rely on the MLP and the multi-level structure to disambiguate collisions. The question is how severely collisions degrade reconstruction quality as the input dimension increases, and whether low-dimensional split encoders provide a practical alternative.

4.2 QuadCubes (Baseline)

The QuadCubes architecture was described in full in section 2.5.3. It uses four 3D multiresolution hash encoders covering all four possible 3-element subsets of $\{x, y, z, t\}$. Every pair of coordinates appears together in at least two encoders, and the spatial encoder `xyz` overlaps with each of the three temporal encoders in two dimensions. With the baseline configuration from Chapter 3 ($L = 23$, $\log_2 T = 23$, $F = 4$), the four encoders occupy approximately 5.2 GB of GPU memory at float16. QuadCubes serves as the primary baseline throughout this chapter. **[TODO: State the specific hashmap size used for QuadCubes in this chapter's experiments, noting whether it matches the Chapter 3 baseline or a reduced size chosen for fair comparison.]**

4.3 SingleCube

The most straightforward extension of the static 3D model to 4D is a single 4D multiresolution hash encoder taking (x, y, z, t) as input and mapping directly to a feature vector for the MLP. This is conceptually the natural scaling of NAF from three to four dimensions.

As established in section 4.1, a collision-free 4D encoder at a finest resolution of 1024 would require a hashmap of $2^{40} \approx 10^{12}$ entries. Any practical 4D encoder must therefore operate with a hashmap many orders of magnitude smaller, resulting in a severe collision load. With $\log_2 T = 23$, the expected collision rate at the finest level is $N_{\max}^4 / T = 2^{40}/2^{23} = 2^{17} \approx 131\,000$ vertices per hash table entry. Each stored feature vector must represent an average over more than 100 000 distinct spatial locations, and the disambiguation mechanism has only the levels of the same encoder to draw on, all of which share the same severe collision regime.

Figure **??** shows representative reconstruction slices from the SingleCube model. **[TODO: Insert figure. These are expected to be very noisy.]**

[TODO: Describe the visual character of the reconstruction artefacts.]

Figure ?? shows quantitative metrics for the SingleCube model compared to QuadCubes. **[TODO: Insert PSNR/SSIM/MAE comparison figure.]**

[TODO: Describe the quantitative metric values.]

The SingleCube experiment demonstrates that the collision rate is not merely a theoretical concern but has a direct and observable effect on reconstruction quality. The disambiguation mechanism that allows the QuadCubes encoders to handle moderate collision loads depends on cross-encoder context that is unavailable in a single-encoder design.

4.4 SexCubes

Moving to the other end of the design space, the SexCubes architecture uses six 2D hash encoders covering all six possible 2-element subsets of $\{x, y, z, t\}$, following the approach taken in K-Planes [Fridovich-Keil et al., 2023] and HexPlane [Cao and Johnson, 2023] for dynamic novel-view synthesis.

The central advantage of 2D encoders is a substantially lower collision rate. A 2D encoder with $\log_2 T = 20$ is collision-free at all resolutions up to 1024×1024. Six such encoders total approximately 768 MB, roughly one eighth the memory of the baseline QuadCubes configuration.

The limitation is that each encoder captures interactions between exactly two coordinates and is entirely blind to higher-order interactions. Reconstructing a voxel at (x, y, z, t) from six plane projections is analogous to recovering a 4D function from its six 2D marginals, which is underdetermined without additional coupling. The MLP must therefore perform a substantially harder integration task than in the 3D encoder architectures.

Figure ?? shows representative reconstruction slices from SexCubes alongside QuadCubes. **[TODO: Insert figure.]**

Figure ?? shows quantitative metrics. **[TODO: Insert PSNR/SSIM/MAE comparison figure.]**

[TODO: Describe the observed quality differences. Are errors concentrated at features requiring cross-dimensional coupling?]

The MLP in the SexCubes experiments was **[TODO: specify: same 4-layer 128-neuron MLP as QuadCubes, or widened to compensate for the increased coupling burden]**. **[TODO: If a larger MLP was tested, describe the results here.]**

4.5 Alternative Decoders

The architectures above all use the same small MLP as the decoder. This section investigates whether a more capable decoder can compensate for the limitations of the encoder, particularly for SexCubes where the MLP bears a heavier integration burden.

4.5.1 Transformer Decoder

[TODO: Describe the transformer decoder architecture. For example, treating each encoder's output as a token and applying self-attention to allow cross-encoder information exchange before producing the final attenuation prediction. Specify the number of layers, heads, and embedding dimension used.]

[TODO: Present results. Does the transformer decoder improve SexCubes quality? Does it help QuadCubes? What is the computational overhead?]

4.5.2 U-Net Decoder

[TODO: Describe the U-Net decoder architecture. For example, a small 1D or 2D U-Net applied to the concatenated encoder features. Specify the architecture details.]

[TODO: Present results. Does the U-Net decoder improve reconstruction quality for any encoder configuration?]

4.6 Dense Encoders

The hash encoding stores features in a fixed-size table and relies on collision disambiguation when the grid is finer than the table. An alternative for low-dimensional encoders where the collision-free table size is practical is to use a dense grid that stores one feature vector per grid vertex, eliminating collisions entirely. For a 2D encoder at resolution 1024×1024 with $F = 4$ features per vertex, a single-level dense grid requires approximately 4 MB. A multi-level dense encoder with L levels requires the sum of grid sizes across all levels.

[TODO: Describe the dense encoder implementation. State which levels are dense and which (if any) remain hashed.]

[TODO: Present results for dense encoders replacing the hash encoders in CombinedCubes or SexCubes. Is there a measurable quality improvement from eliminating collisions in the temporal encoders?]

4.7 CombinedCubes

Informed by the observation from the encoder size experiments in Chapter 3 that non-uniform capacity allocation can be beneficial, and by the collision analysis above, the CombinedCubes architecture uses one 3D encoder covering the purely spatial dimensions `xyz` and three 2D encoders covering the space-time pairs `xt`, `yt`, and `zt`:

$$\hat{\mu}(\mathbf{r}, t) = \mathrm{MLP}_{\Phi}\Big(\big[\gamma_{xyz}(\mathbf{r};\, \theta_1),\ \gamma_{xt}(x, t;\, \theta_2),\ \gamma_{yt}(y, t;\, \theta_3),\ \gamma_{zt}(z, t;\, \theta_4)\big]\Big). \tag{4.1}$$

The motivation is to decouple the representation of static spatial structure from temporal dynamics in a way that matches the physical structure of the problem. The spatial encoder alone handles the time-invariant rock matrix. The three 2D temporal encoders each capture how one spatial coordinate co-varies with time.

The key practical consequence is the hashmap size required for the temporal encoders. A 2D encoder is collision-free at $\log_2 T = 20$ for resolutions up to 1024×1024. At float16 with $L = 16$ levels and $F = 4$ features, each 2D encoder at $\log_2 T = 20$ occupies approximately 128 MB. Three such encoders total approximately 384 MB. The `xyz` encoder at $\log_2 T = 23$ adds a further approximately 1.3 GB. The total hash table memory for CombinedCubes is therefore approximately 1.7 GB, compared to approximately 5.2 GB for the baseline QuadCubes, a reduction of roughly 67%.

Figure **??** shows representative reconstruction slices from CombinedCubes alongside QuadCubes. **[TODO: Insert figure.]**

Figure **??** shows quantitative metrics. **[TODO: Insert PSNR/SSIM/MAE figure.]**

[TODO: Describe the observed quality difference or equivalence between CombinedCubes and QuadCubes.]

Figure **??** shows training time and peak GPU memory consumption for CombinedCubes compared to QuadCubes. **[TODO: Insert figure.]**

[TODO: Describe the training time comparison. Quantify the speedup if possible.]

4.8 Dense Temporal Encoders in CombinedCubes

The CombinedCubes architecture uses 2D hash encoders for the temporal dimensions, which are already collision-free at $\log_2 T = 20$. Replacing these with fully dense encoders eliminates the hash function entirely, removing even the theoretical possibility of collisions and potentially improving gradient flow.

[TODO: Describe the dense variant of CombinedCubes. State the grid resolution and memory cost of the dense temporal encoders.]

[TODO: Present results comparing CombinedCubes with hashed versus dense temporal encoders. This should be evaluated on multiple datasets to confirm the finding is robust.]

[TODO: Compare against NeCT QuadCubes and other baselines (e.g. FDK, OS-SART-TV) on the benchmark datasets to contextualise the results.]

4.9 Comparative Analysis

Table 4.1 summarises the encoder architectures and their properties.

Table 4.1: Dynamic encoder architecture design space. The collision rate column gives a qualitative assessment relative to a collision-free 3D encoder.

Architecture	Encoders	Dim	Coordinate subsets	Collision regime
SingleCube	1	4	$xyzt$	Extreme
QuadCubes	4	3	All 4 possible 3-subsets	Moderate
SexCubes	6	2	All 6 possible 2-subsets	Low
CombinedCubes	4	3+2+2+2	xyz, xt, yt, zt	Low–Moderate

Table 4.2 summarises all architectures across reconstruction quality, memory, and training time. **[TODO: Fill in the table with measured values.]**

Table 4.2: Summary comparison of all encoder architectures on **[TODO: dataset name]**.

Architecture	PSNR (dB)	SSIM	MAE	Memory (GB)	Ti
SingleCube	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[T
QuadCubes	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[T
SexCubes	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[T
CombinedCubes	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[T
CombinedCubes (dense temporal)	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[T

Figure **??** plots all dynamic architectures on a quality-versus-memory plane and a quality-versus-time plane. **[TODO: Insert figures. Mark the QuadCubes baseline prominently.]**

[TODO: Describe the overall pattern in the results. Which architectures are on the Pareto front? Is CombinedCubes with dense temporal encoders the best overall?]

Chapter 5

Continuous Scanning

The acquisition protocols described in previous chapters all assume a step-and-shoot scanning mode, in which the sample is brought to a stop at each angular position before a projection is recorded. An alternative is continuous rotation scanning, sometimes called fly-scan, in which the sample rotates at a constant angular velocity and the shutter remains open for the full duration of the scan. This chapter describes the physical difference between the two modes, derives the modified forward model required to reconstruct data acquired in continuous rotation mode, and presents experimental results evaluating the implementation within NeCTv2.

5.1 Motivation

In step-and-shoot mode the sample is stationary during each exposure, so every photon reaching the detector passed through the sample at the same angular orientation. The cost of this geometric cleanliness is time. Between each exposure the sample must decelerate, the controller waits for vibration to damp out, the shutter opens, the exposure runs, the shutter closes, and the motor accelerates again. For instruments where each exposure is short relative to the motor settling time, a significant fraction of the total acquisition time is spent stationary but not imaging.

In continuous rotation mode the shutter remains open throughout. The sample rotates steadily and the detector integrates photons over the full angular range swept during the exposure time. If the exposure time is τ and the angular velocity is ω, then each projection integrates the sample over an angular interval of $\Delta\phi = \omega\tau$ radians. The resulting projection is geometrically blurred in the angular direction, with severity proportional to $\Delta\phi$.

For dynamic experiments continuous rotation has a direct benefit. Processes that occur faster than the step-and-shoot inter-frame interval may be partially or fully unresolvable in that mode, but if continuous rotation reduces the effective time between adjacent projections those same processes become accessible. The trade-off

is that each projection is individually less geometrically precise, and a reconstruction algorithm that treats them as step-and-shoot projections will see systematic errors in the forward model.

5.2 Forward Model for Continuous Scanning

In step-and-shoot mode the intensity recorded at detector pixel j at angle ϕ is given by Beer–Lambert's law along a single ray:

$$I_j^{\phi} = I_0 \exp\!\left(- \int_{\mathbf{r}_j^{\phi}} \mu(\mathbf{r}(t))\, \mathrm{d}t \right). \tag{5.1}$$

In continuous rotation mode the sample rotates from angle ϕ_s to $\phi_e = \phi_s + \Delta\phi$ during the exposure. Assuming constant angular velocity and linear detector response, the recorded intensity is the time-average of the instantaneous Beer–Lambert transmission:

$$I_j^{[\phi_s,\phi_e]} = \frac{1}{\Delta\phi} \int_{\phi_s}^{\phi_e} I_0 \exp\!\left(- \int_{\mathbf{r}_j^{\phi}} \mu(\mathbf{r}(t))\, \mathrm{d}t \right) \mathrm{d}\phi. \tag{5.2}$$

This integral has no closed form in general. It is approximated by dividing the swept interval into K equal sub-intervals and evaluating the Beer–Lambert transmission at the midpoint of each:

$$\hat{I}_j^{[\phi_s,\phi_e]} = \frac{1}{K} \sum_{k=0}^{K-1} I_0 \exp\!\left(- \sum_p \hat{\mu}\big(\mathbf{r}_p^{\phi_k},\, t_j\big)\, \Delta s \right), \qquad \phi_k = \phi_s + \left(k + \tfrac{1}{2}\right) \frac{\Delta\phi}{K}, \tag{5.3}$$

where $\mathbf{r}_p^{\phi_k}$ are the equidistant sample points along the ray at sub-step angle ϕ_k, Δs is the step size, and t_j is the timestamp of the projection. Setting $K = 1$ recovers the standard single-angle forward model with the ray placed at the midpoint angle, which is equivalent to the step-and-shoot forward model with a possible angular error of up to $\Delta\phi/2$. Increasing K improves the accuracy of the approximation at the cost of K times as many network evaluations per ray per training step.

Algorithm 1 gives the pseudocode for the continuous-scan forward pass implemented in NeCTv2.

The computational cost scales linearly with K relative to the standard single-angle pass. The gradient with respect to the network parameters flows back through each of the K Beer–Lambert integrals independently, so the backward pass also scales linearly with K. In practice the total training time for a continuous-scan reconstruction with K accumulation steps is therefore approximately K times the training time for the equivalent step-and-shoot reconstruction.

Algorithm 1 Continuous-Scan Forward Pass in NeCTv2

Require: Angular interval $[\phi_s, \phi_e]$, accumulation steps K, projection timestamp t, max traversal length $d_{\max}$

1: Compute $K{+}1$ boundary angles $\{\psi_0, \psi_1, \ldots, \psi_K\} \leftarrow \text{linspace}(\phi_s,\ \phi_e,\ K{+}1)$
2: $\hat{y}_j \leftarrow 0$ **for all** detector pixels j
3: **for** $k = 0, \ldots, K-1$ **do**
4: $\phi_k \leftarrow (\psi_k + \psi_{k+1}) \,/\, 2$ ▷ Mid-point of sub-interval k
5: Update source/detector geometry to angle ϕ_k
6: Cast rays; sample P equidistant points $\{\mathbf{r}_p^{\phi_k}\}$ along each ray j within the volume
7: Query network: $\hat{\mu}_p \leftarrow f_\theta\big(\mathbf{r}_p^{\phi_k},\, t\big)$ for $p = 1, \ldots, P$ ▷ Same t for all rays and sub-angles
8: $\hat{y}_j \leftarrow \hat{y}_j + \dfrac{1}{K} \cdot \dfrac{d_j}{d_{\max}} \sum_{p=1}^{P} \hat{\mu}_p$ ▷ d_j: physical traversal length of ray j
9: **end for**
10: **return** $\hat{y}_j$ **for all** j ▷ Predicted line integral; compare directly to log-domain sinogram

Gradient computation (two-pass, memory-efficient):

11: **Pass 1** (no gradients): execute lines 3–9 for all k, cache projector outputs $\{\mathbf{r}_p^{\phi_k}, d_j\}$; obtain $\hat{y}_j$
12: Treat $\hat{y}_j$ as a leaf variable; compute loss $\mathcal{L}$ and back-propagate to obtain $\partial\mathcal{L}/\partial\hat{y}_j$
13: **Pass 2** (with gradients): replay each k using cached outputs; back-propagate $\sum_j (\partial\mathcal{L}/\partial\hat{y}_j) \cdot \text{contrib}_{k,j}$ through f_θ

5.3 Static Dataset Experiments

To validate the continuous-scan forward model in isolation from dynamic reconstruction, it was first applied to a static dataset. A static reconstruction does not require the temporal encoders of QuadCubes and uses the 3D hash-encoded MLP described in section 2.5.2. The dataset used was **[TODO: describe the static dataset: name, resolution, number of projections, and how the continuous-scan data was simulated or acquired]**.

The purpose of the static experiment is not to demonstrate a benefit from continuous scanning, since a static sample does not change between exposures and the angular blurring is a pure artefact rather than meaningful physical information. Instead the static experiment tests whether the forward model correctly accounts for the blurring.

Figure **??** shows representative reconstruction slices for a range of K values at a fixed $\Delta\phi =$ **[TODO: value]**. **[TODO: Insert figure.]**

Figure **??** shows PSNR and SSIM as a function of K for **[TODO: several values of $\Delta\phi$]**. **[TODO: Insert figure.]**

The static results showed that the multi-step accumulation forward model successfully compensates for the angular blurring. At $K = 1$ the reconstruction exhibits artefacts consistent with the angular mismatch. As K increases, quality improves and converges at **[TODO: describe plateau]**. Beyond a certain K the improvement saturates because the midpoint approximation within each sub-interval becomes negligibly small relative to other sources of error. **[TODO: Quantify if possible.]**

5.4 Dynamic Dataset Experiments

The static experiment established that the forward model is correctly implemented. The dynamic experiment extends this to time-resolved reconstruction, where continuous scanning provides a physical advantage over step-and-shoot because the detector captures information continuously and processes that occur within a single exposure interval are reflected in the recorded intensity.

The dataset used was **[TODO: describe the dynamic dataset]**.

[TODO: Describe the reconstruction qualitatively once results are available.]

Figure **??** shows **[TODO: dynamic reconstruction slices comparing $K = 1$ and higher values.]**

[TODO: Describe the observed differences.]

5.5 Discussion

The continuous-scan forward model extends NeCTv2 to a broader class of CT acquisition protocols without requiring any change to the network architecture or training procedure. The only modification is the replacement of the single-ray Beer–Lambert evaluation with the K-step averaged forward pass.

The benefit of continuous scanning is most significant for dynamic experiments where the process being imaged has a characteristic timescale comparable to or shorter than the inter-frame interval of a step-and-shoot acquisition. In that regime, step-and-shoot imaging misses events that occur between frames, while continuous scanning integrates over those events and records their signature in the blurred projection data.

The practical cost is that individual projections are geometrically less precise. Whether this additional ambiguity is outweighed by the increased angular sampling rate depends on the specific sample and process being imaged. **[TODO: Describe any observations from the experimental results that speak to this trade-off.]**

The value of K required for convergence is a practical consideration. The static results suggest that **[TODO: restate the K saturation finding]**. Since training time scales linearly with K, choosing the smallest adequate K is important.

[TODO: Describe any remaining limitations, e.g. whether the model assumes a fixed angular velocity or handles alternating rotation direction.]

Chapter 6

Conclusion

6.1 Summary of Contributions

[TODO: Bullet-point recap of what was achieved in each chapter.]

6.2 Limitations and Future Work

[TODO: Remaining open questions: spatiotemporal resolution quantification, hyperparameter sensitivity, training time, extension to polychromatic sources, parallel-beam synchrotron settings, energy-resolved CT.]

Bibliography

A. H. Andersen and A. C. Kak. Simultaneous algebraic reconstruction technique (SART): A superior implementation of the ART algorithm. *Ultrasonic Imaging*, 6 (1):81–94, 1984. doi: 10.1177/016173468400600107.

Ang Cao and Justin Johnson. HexPlane: A fast representation for dynamic scenes. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 130–141, 2023.

Guang-Hong Chen, Jie Tang, and Shuai Leng. Prior image constrained compressed sensing (PICCS): a method to accurately reconstruct dynamic CT images from highly undersampled projection data sets. *Medical Physics*, 35(2):660–663, 2008. doi: 10.1118/1.2836423.

George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989. doi: 10.1007/BF02551274.

Yang Du, Gao Yu, Xia Xiang, Xiaoming Wang, and Yves De Deene. Convergence of SART + OS + TV iterative reconstruction algorithm for optical CT imaging of gel dosimeters. *Journal of Physics: Conference Series*, 847:012025, 2017. doi: 10.1088/1742-6596/847/1/012025.

L. A. Feldkamp, L. C. Davis, and J. W. Kress. Practical cone-beam algorithm. *Journal of the Optical Society of America A*, 1(6):612–619, 1984. doi: 10.1364/JOSAA.1.000612.

Sara Fridovich-Keil, Giacomo Meanti, Frederik Warburg, Benjamin Recht, and Angjoo Kanazawa. K-Planes: Explicit radiance fields in space, time, and appearance. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12479–12488, 2023.

Henrik Friis, Håkon Nese, Giacomo Luani, Colin Pryme, Lars Rennan, Basab Chattopadhyay, Anders Kristoffersen, Ole Jakob Mengshoel, and Dag Werner Breiby. Implicit neural representation for fast 4D computed tomography of

multiphase flow in porous media. *Communications Physics*, 8:339, 2025. doi: 10.1038/s42005-025-02249-0.

Wout Goethals et al. Dynamic CT reconstruction with improved temporal resolution for scanning of fluid flow in porous media. *Water Resources Research*, 58: e2021WR031365, 2022. doi: 10.1029/2021WR031365.

Jeff Gostick et al. PoreSpy: A Python toolkit for quantitative analysis of porous media images. *Journal of Open Source Software*, 4(37):1296, 2019. doi: 10.21105/joss.01296.

Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989. doi: 10.1016/0893-6080(89)90020-8.

Dianlin Hu et al. Hybrid-domain neural network processing for sparse-view CT reconstruction. *IEEE Transactions on Radiation and Plasma Medical Sciences*, 5 (1):88–98, 2021. doi: 10.1109/TRPMS.2020.3011413.

Kyong Hwan Jin, Michael T. McCann, Emmanuel Froustey, and Michael Unser. Deep convolutional neural network for inverse problems in imaging. *IEEE Transactions on Image Processing*, 26(9):4509–4522, 2017. doi: 10.1109/TIP.2017.2713099.

James T. Kajiya and Brian P. Von Herzen. Ray tracing volume densities. In *Proceedings of the 11th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, pages 165–174, 1984. doi: 10.1145/800031.808594.

Avinash C. Kak and Malcolm Slaney. *Principles of Computerized Tomographic Imaging*. Society for Industrial and Applied Mathematics, Philadelphia, 2001.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

Pavol Klacansky. Open SciVis datasets. `https://klacansky.com/open-scivis-datasets/`, 2017.

Thomas Kohler. A projection access scheme for iterative reconstruction based on the golden section. In *IEEE Symposium Conference Record Nuclear Science 2004*, volume 6, pages 3961–3965, 2004. doi: 10.1109/NSSMIC.2004.1462462.

Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.

Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021. doi: 10.1145/3503250.

Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Transactions on Graphics*, 41(4):102:1–102:15, 2022. doi: 10.1145/3528223.3530127.

Glenn R. Myers, Andrew M. Kingston, Trond K. Varslot, Matthew L. Turner, and Adrian P. Sheppard. Dynamic tomography with a priori information. *Applied Optics*, 50(20):3685–3690, 2011. doi: 10.1364/AO.50.003685.

Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pages 807–814, 2010.

Mayank Patwari et al. Reducing the risk of hallucinations with interpretable deep learning models for low-dose CT denoising: comparative performance analysis. *Physics in Medicine & Biology*, 68(19):19LT01, 2023. doi: 10.1088/1361-6560/acf5d0.

Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred A. Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 5301–5310, 2019.

Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241. Springer, 2015.

David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986. doi: 10.1038/323533a0.

Vincent Sitzmann, Julien N.P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems*, volume 33, pages 7462–7473, 2020.

Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional

domains. In *Advances in Neural Information Processing Systems*, volume 33, pages 7537–7547, 2020.

Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004. doi: 10.1109/TIP.2003.819861.

Philip J. Withers et al. X-ray computed tomography. *Nature Reviews Methods Primers*, 1:1–17, 2021. doi: 10.1038/s43586-021-00015-4.

Ruyi Zha, Yanhao Zhang, and Hua Li. NAF: Neural attenuation fields for sparse-view CBCT reconstruction. In *International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 442–452. Springer, 2022.

Appendix A

Supplementary Experimental Details

[TODO: Detailed hyperparameter tables, full benchmark results, extra figures.]

Appendix B

NeCTv2 Hyperparameter Reference

[TODO: Complete listing of all hyperparameters used in each experiment.]